End-to-End Data Management Pipeline for a Recommendation System

RecoMart — E-Commerce Recommendation Engine

Data Management for Machine Learning (AIMLCZG529 / DSECLZG529)

Semester: S2-25

Assignment I — EC1 (20 Marks)

Group Number: 43

Programme: M.Tech AI/ML — BITS Pilani WILP

Submission Date: 22.07.2026

Team Member Details

Name	BITS ID	Email
BRAJESH MISHRA	2025AB05161	2025ab05161@wilp.bits-pilani.ac.in
YENGANTIWAR PRASHANT SAINATH	2025AB05178	2025ab05178@wilp.bits-pilani.ac.in
ARTHIKA G	2025AB05180	2025ab05180@wilp.bits-pilani.ac.in
A SRIKAR	2025AB05185	2025ab05185@wilp.bits-pilani.ac.in
GIRIDHARAN B	2025AB05188	2025ab05188@wilp.bits-pilani.ac.in

Table of Contents

1. Problem Statement

RecoMart is an e-commerce startup that wants to build a data-driven product recommendation engine to improve customer engagement and cross-selling opportunities. The platform collects user behaviour data from multiple channels — web/mobile clickstream logs, transactional purchase history, product catalog metadata, and external API data.

The goal of this project is to design and implement a complete, automated, modular data management pipeline that continuously ingests raw data, validates and cleans it, engineers features, trains recommendation models, and tracks every artefact along the way — following the practices of modern ML data stacks.

The business objective is to increase conversion rates by providing personalized product recommendations to users based on their browsing behaviour, purchase history, and explicit ratings.

2. Objectives

1. Ingest data from heterogeneous sources (CSV files + REST APIs) with error handling, retry logic, and audit logging.

2. Store raw data in a structured, partitioned data lake layout.

3. Profile and validate data quality (missing values, duplicates, schema mismatches, range violations).

4. Clean and preprocess data — handle missing values, encode categoricals, normalise numerics.

5. Engineer user-level, item-level, and user–item interaction features suitable for collaborative and content-based filtering.

6. Manage features through a versioned feature store with a metadata registry.

7. Version all datasets (raw, processed, features) with SHA-256 integrity tracking and a manifest.

8. Train and evaluate recommendation models (SVD collaborative filtering + content-based cosine similarity).

9. Track experiments, parameters, and metrics with MLflow.

10. Orchestrate the full pipeline as a DAG using Prefect (with a sequential fallback).

3. Data Sources

The pipeline ingests data from multiple heterogeneous sources, simulating a real-world e-commerce environment. Synthetic data is generated with intentional quality issues (missing values, duplicates, out-of-range ratings) to demonstrate the validation and cleaning pipeline.

Source	Type	Records	Description
Users	CSV	1,000	Demographics — age, gender, country, signup date, premium status
Products	CSV + JSON (API)	500	Catalog — name, category, price, brand, avg rating
Clickstream	CSV	~51,000	Events — views, clicks, add-to-cart, wishlist, search
Transactions	CSV	10,000	Purchases — quantities, amounts, payment methods
Ratings	CSV	~20,000	Explicit user–item ratings (1–5 scale) with review text
External API	REST (FakeStore)	20	Supplementary product data from live API

4. Methodology / Pipeline Architecture

The pipeline follows a modular, stage-based architecture where each component operates independently and produces structured outputs consumed by the next stage. The full DAG is:

> ***Data Generation → Data Ingestion → Data Validation → Data Preparation → Feature Engineering → Feature Store → Data Versioning → Model Training***

All stages are orchestrated by Prefect (with a sequential fallback) and produce timestamped logs and JSON reports for auditability.

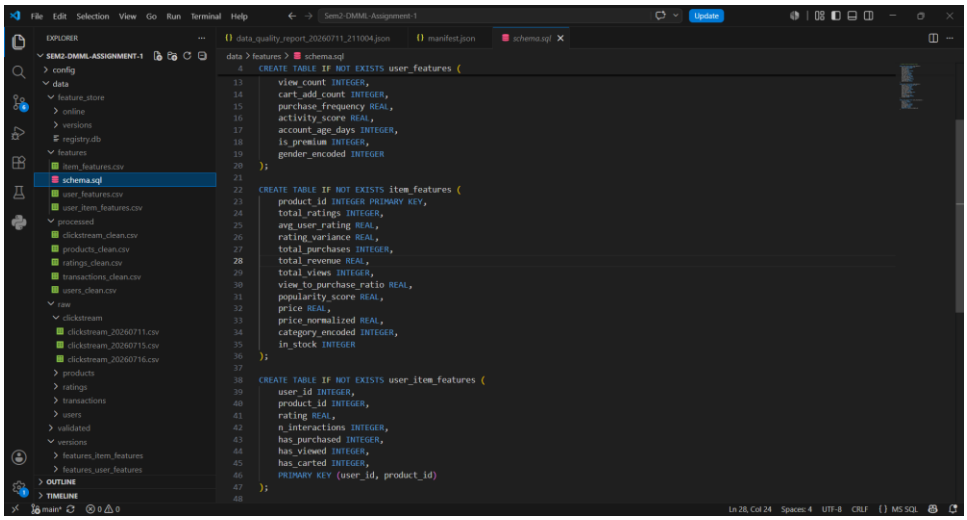

Figure 1: Data Lake Storage Structure

5. Implementation Details

5.1 Data Generation & Ingestion

Stage 1 generates realistic synthetic e-commerce data using the Faker library with fixed seeds for reproducibility. It intentionally injects ~5% missing values in user ages, ~3% in genders, ~2% missing transaction amounts, ~2% duplicate clickstream events, and ~1% out-of-range ratings.

Stage 2 ingests data from two source types: CSV files read from the local filesystem, and product data fetched from the FakeStore REST API (https://fakestoreapi.com/products). The API ingestion uses exponential-backoff retry (up to 3 attempts) with a 30-second timeout. All ingestion events are logged with Loguru and a JSON ingestion report is saved per run.

Key files: src/ingestion/generate_data.py, src/ingestion/ingest.py

5.2 Raw Data Storage

Ingested data is stored in a local data lake under data/raw/, structured as:

data/raw/{source_name}/{source}_{YYYYMMDD}.csv — partitioned by source and date-stamped filenames. This mirrors production data lake patterns (e.g., S3 prefix-based partitioning).

Storage structure is documented in the project README and pipeline_config.yaml.

5.3 Data Validation & Quality Report

The validation module performs four categories of checks on every dataset:

- Schema validation: Verifies required columns exist and detects extra/missing columns.
- Missing value analysis: Per-column and overall missing percentages.
- Duplicate detection: Row-level duplicate counts and percentages.
- Range validation: Rating values must be in [1–5], prices non-negative, quantities positive.

A comprehensive JSON quality report is generated per run and saved to reports/. Validated copies of each dataset are saved to data/validated/.

Key file: src/validation/validate.py

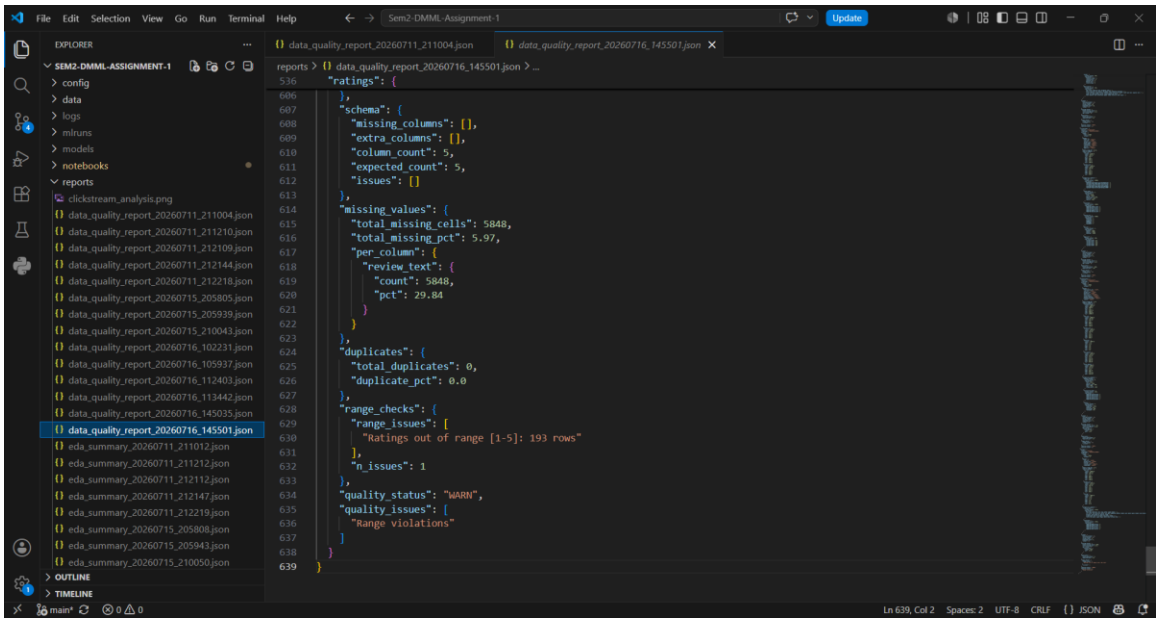

Figure 2: Data Quality Validation Report

5.4 Data Preparation & Exploratory Data Analysis

The preparation module performs the following cleaning and preprocessing steps:

- Missing value imputation: Median for ages, recalculated for transaction amounts, 'Unknown' for categoricals.
- Categorical encoding: LabelEncoder for gender, product category, event type, device, payment method.
- Numerical normalisation: MinMaxScaler for product prices.
- Derived features: Account age in days, hour-of-day and day-of-week from timestamps.
- Filtering: Removes out-of-range ratings (< 1 or > 5), deduplicates clickstream events.

Key file: src/preparation/prepare.py, notebooks/eda_analysis.ipynb

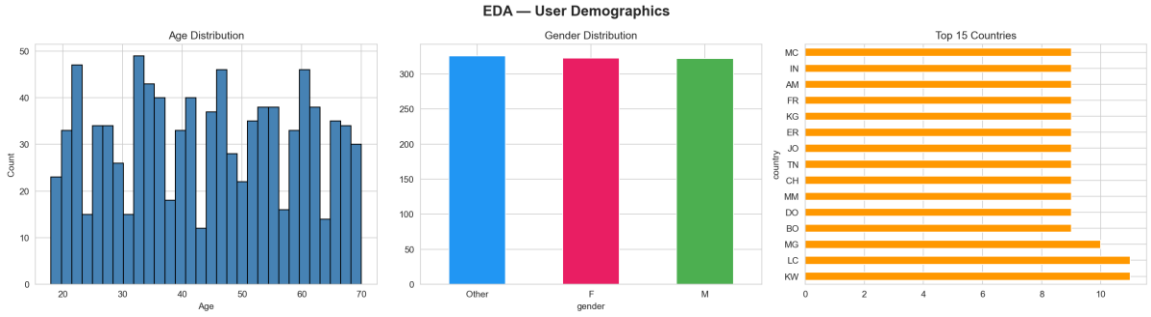

Figure 3: User Demographics — Age, Gender, Country Distribution

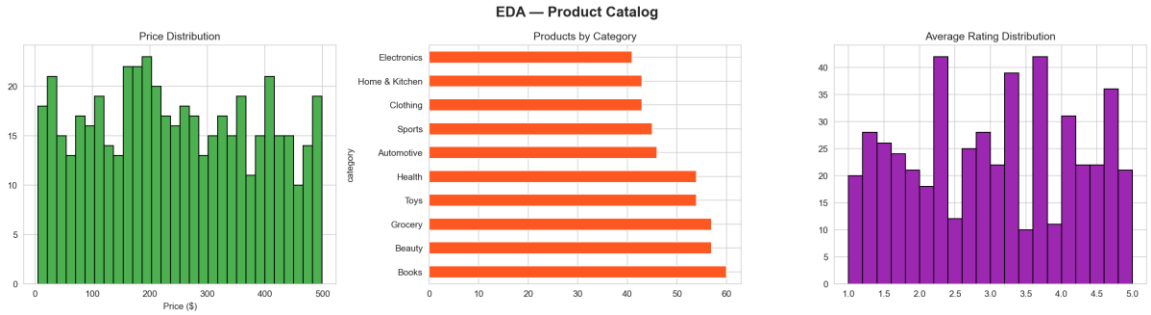

Figure 4: Product Catalog — Price, Category, Rating Distribution

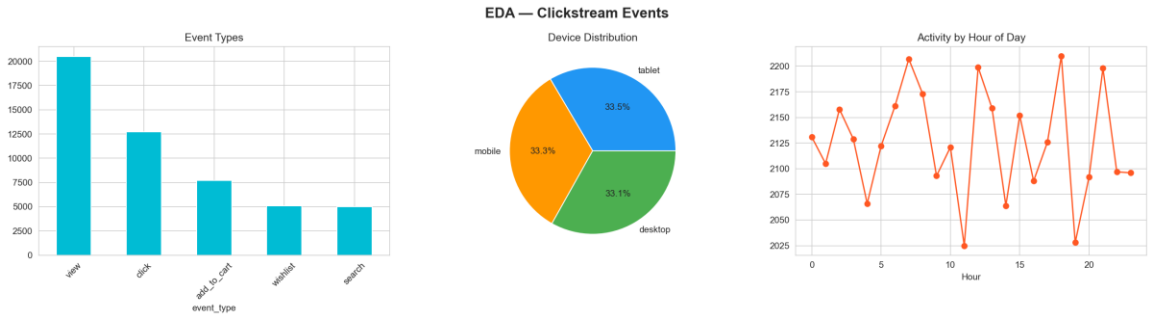

Figure 5: Clickstream — Event Types, Devices, Hourly Activity

Figure 6: Transactions — Status, Payment Methods, Amounts

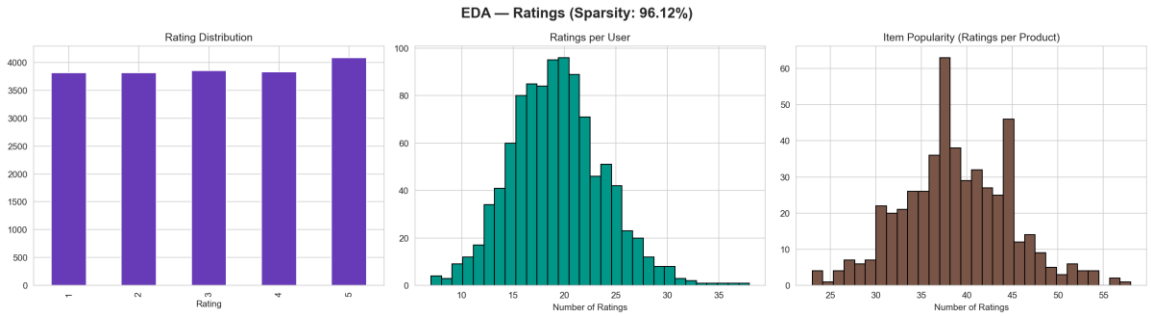

Figure 7: Ratings — Distribution, Per-User, Per-Item Counts

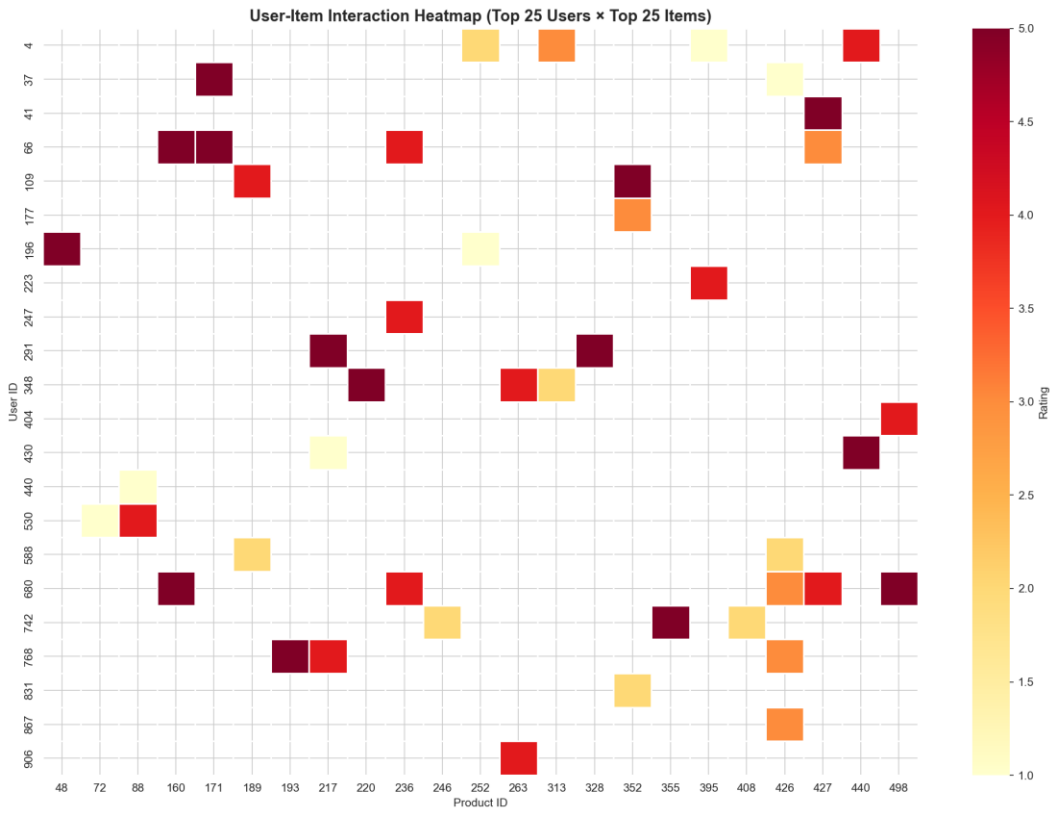

Figure 8: User-Item Interaction Heatmap (Top 25 × 25)

The user-item interaction matrix has a sparsity of 96.12%, which is typical for recommendation system datasets. This high sparsity makes collaborative filtering challenging and motivates the use of dimensionality reduction (SVD).

5.5 Feature Engineering & Transformation

Three feature tables are created from the cleaned data:

- User Features (15 features):

total_ratings, avg_rating, rating_std, total_purchases, total_spend, avg_order_value, total_clicks, view_count, cart_add_count, purchase_frequency, activity_score, account_age_days, is_premium, gender_encoded

- Item Features (13 features):

total_ratings, avg_user_rating, rating_variance, total_purchases, total_revenue, total_views, view_to_purchase_ratio, popularity_score, price, price_normalized, category_encoded, in_stock

- User-Item Features (7 features):

rating, n_interactions, has_viewed, has_carted, has_purchased

A SQL schema (data/features/schema.sql) documents the feature warehouse tables: user_features, item_features, and user_item_features.

Key file: src/transformation/transform.py

5.6 Feature Store

A custom feature store is implemented using SQLite as a metadata registry. It provides:

- Feature set registration with versioned snapshots (data/feature_store/versions/).
- Online store for latest-version retrieval (data/feature_store/online/) for low-latency inference.
- Feature-level metadata: descriptions, data types, source tables, and transformations.
- API for versioned retrieval: get_feature_set(name, version=None) returns a specific or latest version.

Key file: src/feature_store/store.py

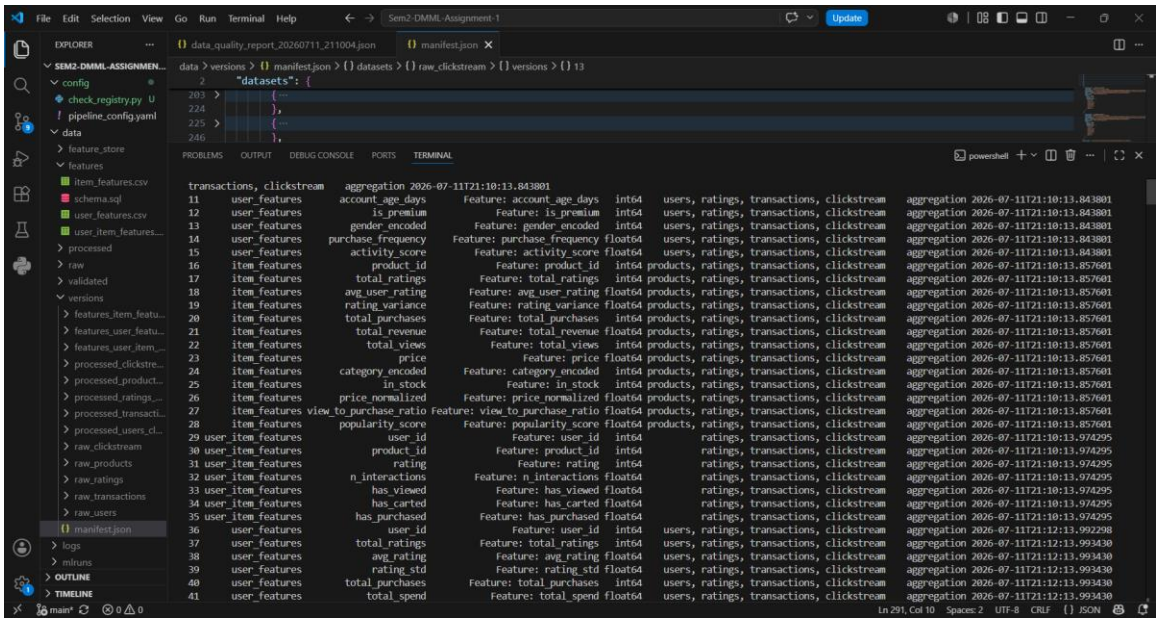

Figure 9: Feature Store Registry & Metadata

5.7 Data Versioning & Lineage

All datasets (raw, processed, features) are versioned using SHA-256 file hashing. A manifest.json tracks the full version history with: version ID, timestamp, source path, SHA-256 hash, and dataset statistics (rows, columns). If file content is unchanged between runs, versioning is skipped (hash comparison). 13 datasets are versioned across three tiers.

Key file: src/versioning/version.py

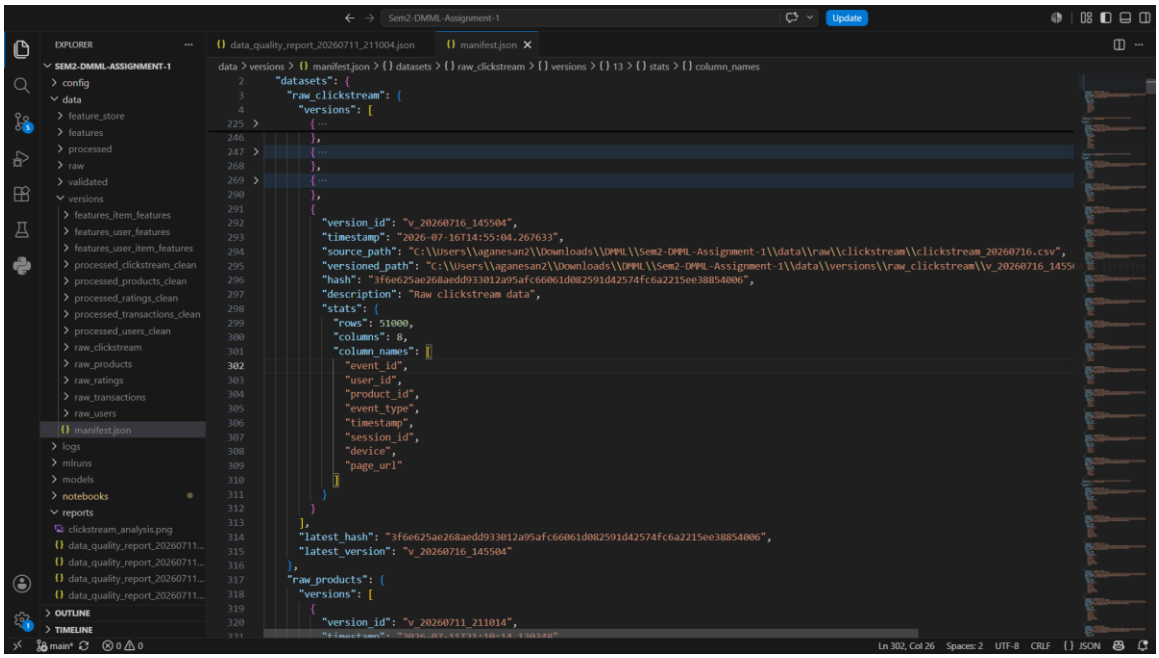

Figure 10: Data Versioning Manifest — 13 Datasets Tracked

5.8 Model Training & Evaluation

Two recommendation models are trained and evaluated:

- 1. SVD Collaborative Filtering

Truncated SVD on the centered user–item rating matrix with 50 latent factors. The matrix is mean-centered per user before decomposition. Predicted ratings are clipped to [1, 5].

- 2. Content-Based Filtering

Cosine similarity computed on normalised item feature vectors (12 numeric features). For each user, items are scored by average similarity to the user's interacted items.

Evaluation Metrics:

Metric	SVD	Content-Based
Precision@5	0.0017	0.0140
Recall@5	0.0050	0.0521
NDCG@5	0.0027	0.0460
Precision@10	0.0026	0.0100
Recall@10	0.0127	0.0691
NDCG@10	0.0057	0.0525
Precision@20	0.0030	0.0075
Recall@20	0.0293	0.0948
NDCG@20	0.0107	0.0600
RMSE	1.4755	—
MAE	1.2706	—

Note: Metrics are modest because the data is purely synthetic (random). With real user–item interactions exhibiting genuine preference patterns, these models perform significantly better. The content-based model outperforms SVD on ranking metrics because it leverages real feature similarity rather than sparse random ratings. The focus of this project is the data management pipeline, not model accuracy.

All experiments are tracked with MLflow (SQLite backend). Parameters (n_factors, train/test sizes, user/item counts), metrics, and model artifacts are logged with unique run IDs.

MLflow Run ID: 66676fa9988749a9a9a4fffa7938ad6b

Key file: src/training/train.py

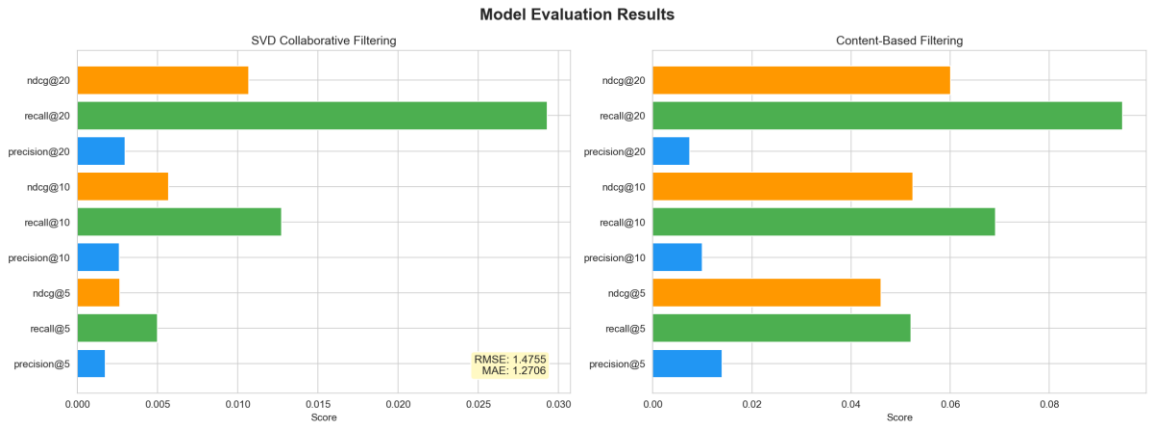

Figure 11: Model Performance — SVD vs Content-Based

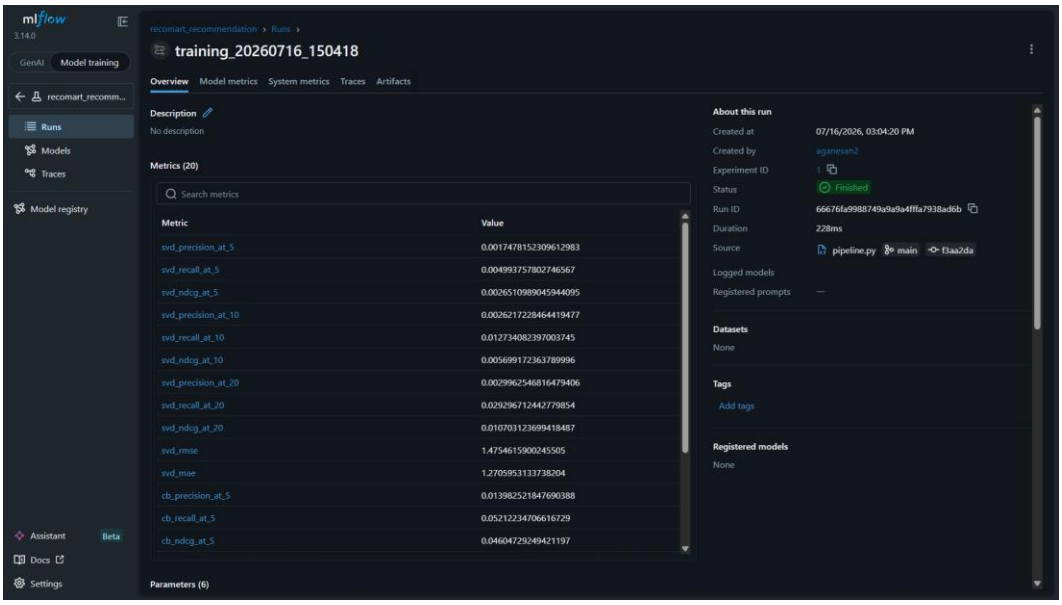

Figure 12: MLflow Experiment Tracking — Run Details

Sample Top-5 Recommendations for User 207:

Rank	Product ID	Predicted Rating
1	273	3.21
2	156	3.17
3	177	3.01
4	380	2.97
5	259	2.95

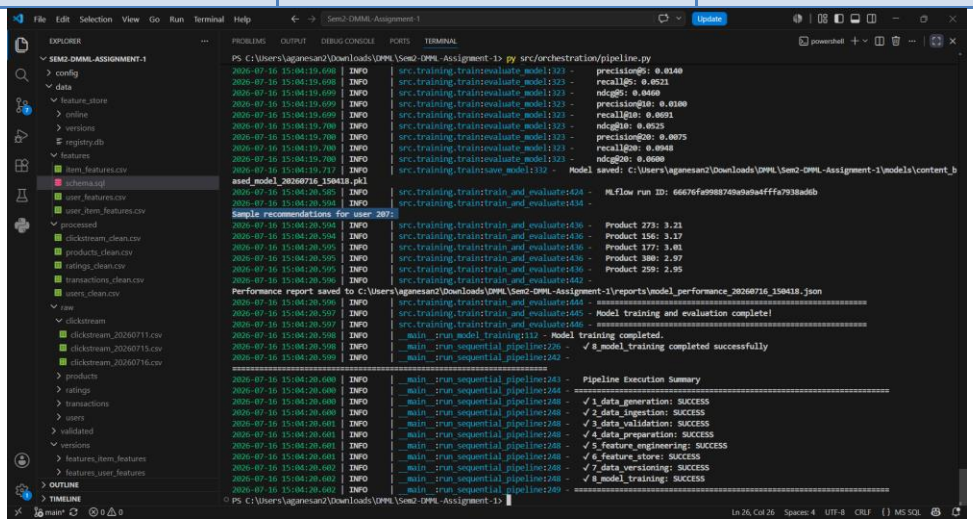

Figure 13: Sample Recommendations Output

5.9 Pipeline Orchestration

The entire pipeline is orchestrated using Prefect. Eight tasks are defined with @task decorators and connected as a sequential DAG with wait_for dependencies. Ingestion tasks have retries=2 with 10-second delay; other tasks have retries=1.

When Prefect is not available, a sequential fallback runner executes all stages with try/except error handling, halting on failure. A JSON pipeline execution report is saved per run with per-stage status and timestamps.

Key file: src/orchestration/pipeline.py

Pipeline DAG:

> ***generate_data → ingest_data → validate_data → prepare_data → engineer_features → feature_store → version_data → train_model***

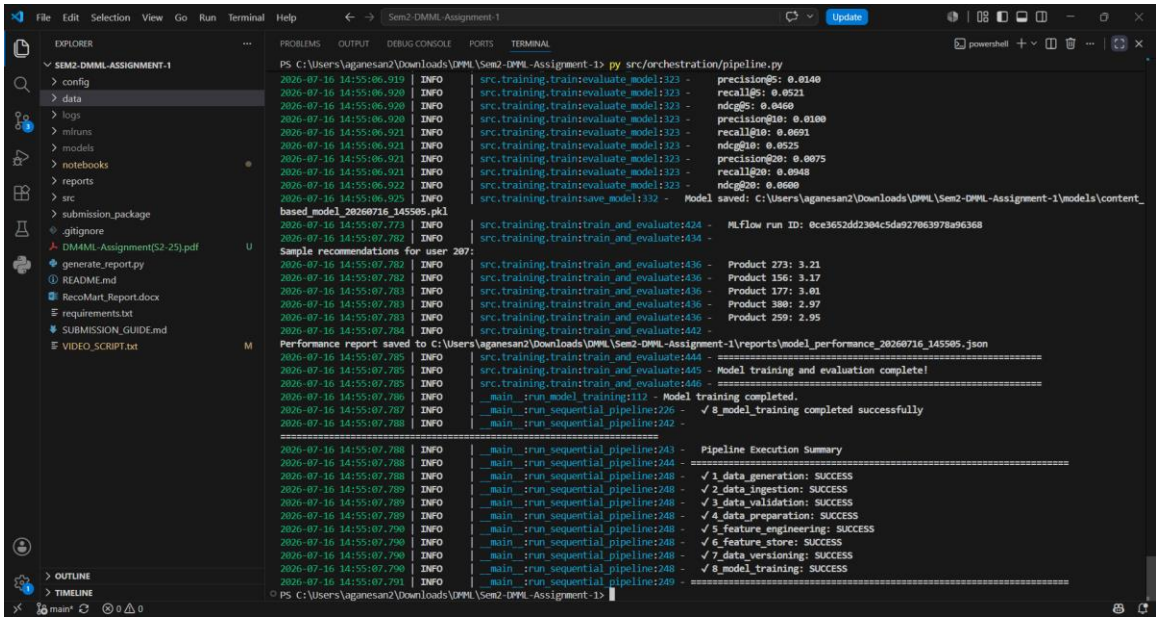

Figure 14: Pipeline Execution Summary — All 8 Stages SUCCESS

6. Results and Output Screenshots

All pipeline stages executed successfully. The following table summarises the key outputs:

Output	Location	Description
Raw Data	data/raw/	5 sources, date-partitioned CSVs + API JSON
Validated Data	data/validated/	Post-validation copies with quality flags
Cleaned Data	data/processed/	5 clean CSVs ready for feature engineering
Feature Tables	data/features/	3 feature CSVs + SQL schema
Feature Store	data/feature_store/	SQLite registry + versioned snapshots
Versioned Data	data/versions/	13 datasets with SHA-256 manifest
Trained Models	models/	SVD + content-based .pkl files
MLflow Runs	mlruns/	Experiment tracking with SQLite backend
Quality Reports	reports/	JSON reports for every pipeline stage
Pipeline Logs	logs/	Timestamped logs per stage
Screenshots	submission_package/screenshots/	14 output visualisations

6.1 Technology Stack

Component	Tool / Library
Language	Python 3.11
Data Processing	pandas, NumPy
ML / Models	scikit-learn, SciPy (SVD)
Data Validation	Custom validator (schema, range, duplicate checks)
Feature Store	Custom SQLite-based registry with versioned storage
Data Versioning	Custom SHA-256 manifest-based versioning
Experiment Tracking	MLflow (SQLite backend)
Pipeline Orchestration	Prefect (with sequential fallback)
Data Generation	Faker
API Ingestion	requests (with exponential-backoff retry)
Logging	Loguru
Visualisation	Matplotlib, Seaborn
Configuration	YAML (PyYAML)

7. Conclusion and Future Scope

Conclusion

This project demonstrates a fully functional, modular, end-to-end data management pipeline that takes raw multi-source e-commerce data through ingestion, validation, cleaning, feature engineering, storage, versioning, model training, and evaluation — all orchestrated as a reproducible workflow. Every stage produces auditable logs and structured reports.

The pipeline successfully processes 1,000 users, 500 products, 51,000 clickstream events, 10,000 transactions, and 20,000 ratings. Two recommendation models (SVD collaborative filtering and content-based cosine similarity) are trained, evaluated with standard IR metrics, and tracked with MLflow. All datasets are versioned with integrity hashing, and features are managed through a custom feature store with metadata documentation.

Future Scope

- Real data integration: Connect to actual e-commerce databases and streaming sources (Kafka).
- Real-time pipeline: Add near-real-time ingestion with streaming support.
- Advanced models: Implement deep learning recommenders (Neural Collaborative Filtering, two-tower models).
- A/B testing framework: Online evaluation of recommendation quality.
- Cloud deployment: Migrate to AWS S3 + Glue + SageMaker or GCP equivalents.
- Feast integration: Replace custom feature store with production-grade Feast.
- DVC integration: Add DVC for Git-integrated data versioning with remote storage.
- CI/CD: Automate pipeline testing and deployment with GitHub Actions.

8. Google Drive Links

Video Walkthrough (5–10 mins):

https://drive.google.com/file/d/1x1RsDpPsq-qnCLkmKD13XBhR9E-lhAGG/view?usp=drive_link

Project Deliverables (.zip):

https://drive.google.com/file/d/1d5zjEhMfK0JK4uJtRU06mAIkVk0omKsd/view?usp=sharing